Från krav till leverans

nedbrytning av krav och estimering

Agenda

- Från kravtext till User Stories (US)
- Från US till Tasks
- Acceptance Criteria
- Definition of Done
- Estimering

> *Material:*
> *Kurssite, User Stories & Tasks ink videoklipps*
> *Kurslitteratur, kap 3*
> *Övning kurssite sprint 1- 'happy path'*

Från spec till kod

- Skapa er en bild av kravspecifikationen.
- Kravspecen berättar `vad` CoEd ska göra, men inte `hur`.

> ***Er uppgift som grupp är att tolka kraven och bryta ned dem i hanterbara delar — så att ni vet vad ni ska koda.***

Tre steg/nivåer av nedbrytning

1. **Kravspecifikation**
2. **User Stories** (användarperspektiv)
3. **Tasks** (det konkreta, inkl ev detaljer, arbetet)

Samt två sätt för att säkerställa kvalité:

- **Acceptance Criteria**
  - *När är US färdigställd?*
  - *Är funktionen rätt?*
- **Definition of Done**
  - *Kvalitetssäkring i projektet*
  - *Är koden byggd på rätt sätt?*

User Stories

Formatet:

> *"Som `<roll>` vill jag `<göra något>` så att `<värde/syfte>`."*

Tre frågor en story ska svara på:

- **Vem** är användaren?
- **Vad** är funktionen?
- **Varför** behövs det?

Varför detta format?

För att lyfta fram och beskriva **värdet** och **funktionaliteten** — *inte bara* tekniken.

No - *"Implementera JWT-autentisering"*

Yes - *"Som användare vill jag kunna logga in så att mina filer är skyddade."*

Dokumentera era US (och tasks)

- Skriv in US i ert projektverktyg (GitHub Projects / Trello / Clickup eller t ex Excel/Google Sheets).
- Det skapar en **Backlog** på ett "naturligt" sätt.

Namngivning och spårbarhet

Varje US kan ges ett ID på formen `US-XNN` :

- **X** = sprintnummer
- **NN** = löpnummer inom sprinten

ID	Krav i kravspec
`US-101`	Krav 2.2.2 Lista filer
`US-102`	Krav 2.2.2 Lista filer
`US-201`	Krav 2.3.1 Kodredigering

Ange alltid vilket krav i kravspecen storyn härrör från. Det skapar **spårbarhet** och *ordning och reda*.

Ett exempel från kravspecen

Krav 2.2.2 Lista filer (Prioritet: Hög)

Användare ska kunna se en översikt över alla sina filer för att enkelt hitta och öppna dem.

- Användare ska se alla sina filer (ägda och delade)
- Listan ska visa relevant metadata (titel, språk, senast ändrad, skapare)
- Listan ska vara sorterbar och eventuellt sökbar
- Användare ska kunna öppna filer genom att klicka på dem
- Delade filer ska ha visuell indikation

Vilket krav väljer vi?

- Utifrån Prioritet
  - **Hög**, medel, låg.
- Finns koppling till **MVP**?
  - I detta fallet ja, eftersom att användare *ska kunna se sina filer*.
- Utifrån vad som bedöms vara rimligt/nödvändigt:
  - vad som tidigare är implementerat (eller inte implementerat).
  - behövs implementeras.

Vi går nu vidare med en "analys" av kravtexten.

Det handlar inte om att omvandla varje punkt i kravtexten till en US.
Det handlar snarare om att försöka se ett kontext där saker och funktionalitet hänger ihop.

Formulera US

- En US beskriver *samma funktion* (som i kravtexten), fast ur ett användarperspektiv:
- `vem` som behöver det och `varför`.
- Som exempel utgår vi från krav 2.2.2, beskriver funktionalitet ur ett systemperspektiv: *vad systemet ska göra*.

Steg 1 - vem?

Läs kravet utifrån — ***vem är det egentligen som vill ha eller behöver detta***?

```
Användare ska kunna se en översikt över alla sina filer för att enkelt hitta och öppna dem.
```

Ett rimligt svar:

en inloggad användare som har filer och behöver se/hitta dem.

Detta ger svaret på **vem**.

Steg 2 - en eller flera funktioner?

Utifrån kravtexten - ***beskriver den en eller flera saker.***

2.2.2 listar 5 punkter:

```
- Användare ska se alla sina filer (ägda och delade)
- Listan ska visa relevant metadata (titel, språk, senast ändrad, skapare)
- Listan ska vara sorterbar och eventuellt sökbar
- Användare ska kunna öppna filer genom att klicka på dem
- Delade filer ska ha visuell indikation
```

Det ser ut som ett krav, men i praktiken handlar det om kanske upp till sex distinkta funktioner:

1. att se sina filer
2. att läsa metadata om filerna
3. att öppna filerna
4. att kunna sortera filer i en lista
5. *eventuellt* sökbar lista över filer
6. att delade filer ska indikeras visuellt

Forts. steg 2

Notera särskilt kravtexten som säger "*eventuellt sökbar*".
Detta indikerar att den funktionen inte är lika högt prioriterad.

> **Det är just den här analysen som är kärnan i arbetet:**
> det handlar om att förstå vad som hänger ihop naturligt och vad som kan separeras.

Forts. steg 2

```
1. Användare ska se alla sina filer (ägda och delade)
2. Listan ska visa relevant metadata (titel, språk, senast ändrad, skapare)
3. Listan ska vara sorterbar och eventuellt sökbar
4. Användare ska kunna öppna filer genom att klicka på dem
5. Delade filer ska ha visuell indikation
```

Utifrån vår analys väljer vi att skapa 5st US :

- En US: *Vi 'slår ihop' nedan två delar - onödigt att dela upp dem separat. Hög prio att göra i kommande sprint;*
  - **se/lista filer (1)**
  - **metadata (2)**
- Två US: en för **sorterbar lista (3)** och en för **"*eventuell sökning*"**.
  - *Fillista implementeras med den första och **eventuell** sökning kan separeras i en egen US med lägre prio.*
- En US: **att öppna en fil (4)** *är en tydlig funktionalitet. Hög prio.*
- En US: **delade filer (5)** *blir en separat US. Göras i en senare sprint. Låg/medel prio.*

Forts. steg 2

> *En praktisk tumregel: om flera saker kan lösas av **samma kodbas/komponent och samma API-anrop, hör de troligtvis ihop i en story**.*
> *Om en sak kräver separat implementation och **kan levereras oberoende av de andra, är det en egen story**.*

Forts. steg 2

Till sist är det värt att stanna upp vid det som **kravtexten inte säger explicit men som ändå måste funka**:

- vad händer om fil-listan är tom?
- vad händer om en delad fil tas bort av ägaren?

Dessa ***edge cases*** måste inte hanteras i separata US utan i acceptance criteria eller i separata tasks — men det är under story-arbetet de brukar dyka upp.

Detta är ett tecken på att nedbrytningen görs på rätt sätt.

Steg 3 - formulera US

Efter ytterligare analys och diskussion landar vi till slut i följande 4 US:

US-101 - Krav: 2.2.2 - Prioritet: Hög

"Som inloggad användare vill jag se en fillista med metadata för varje fil (titel, språk, senast ändrad, skapare), så att jag kan bedöma vilken fil jag vill öppna utan att klicka in i den."

US-102 - Krav: 2.2.2 - Prioritet: Hög

"Som inloggad användare vill jag kunna öppna en fil genom att klicka på den i listan så att jag snabbt kommer till editorn."

forts. Steg 3

US-103 - Krav: 2.2.2 - Prioritet: Medel

"Som inloggad användare vill jag kunna sortera fillistan så att jag hittar rätt fil snabbare."

US-104 - Krav: 2.2.2 - Prioritet: Låg

"Som inloggad användare vill jag kunna söka bland mina filer så att jag slipper scrolla i en lång lista."

forts. Steg 3

Anledningen till att vi själva satte prioritet från hög till låg, hade att göra med deras inbördes beroenden;

fillista till exemepel (US-101), behövde implementeras före t ex sökning och sortering.

Sammanfattning US

- **Analysera** och **bedöm** kravet
  - Prioritet, MVP
  - Formen för US: vem, vad, varför?
  - Ett krav kan ge upphov till **flera** US.
- Välj **inte** bort lågprioriterade stories — planera i så fall in dem i senare sprintar.

Det finns inget facit.

Acceptance Criteria (AC)

Vad är Acceptance Criteria (AC)?

- AC tillhör en **user story** — inte en task.
- AC definierar **när en user story är klar** ur ett funktionellt perspektiv:
  - har vi byggt rätt funktion, och fungerar den som användaren förväntar sig?

De måste vara **testbara påståenden** — antingen stämmer det, eller så stämmer det inte. Undvik vaga formuleringar som "fungerar korrekt" eller "ser bra ut".

> *AC svarar på frågan: gör funktionen rätt sak enligt användaren?*

AC hör till US — inte till tasks

User Story + AC

Storyn beskriver funktionen ur användarens perspektiv.
AC definierar när den storyn är uppfylld.

Task

Ett konkret tekniskt arbetssteg. En task är antingen gjord eller inte.

En task har ingen egen AC.
Däremot kan den ha en kort beskrivning eller checklista av vad "klar" innebär för just den uppgiften.

AC för US-101

US-101 - Krav: 2.2.2 - Prioritet: Hög

"Som inloggad användare vill jag se en fillista med metadata för varje fil (titel, språk, senast ändrad, skapare), så att jag kan bedöma vilken fil jag vill öppna utan att klicka in i den."

Acceptance Criteria:

- Inloggad användare ser en lista med alla sina filer
- Varje fil visar titel, språk, senast ändrad och skaparens namn
- Filer delade med användaren visas med en tydlig visuell markering
- Tom lista visas med ett informativt meddelande
- Ej inloggad användare omdirigeras till login

AC för US-102

US-102 - Krav: 2.2.2 - Prioritet: Hög

"Som inloggad användare vill jag kunna öppna en fil genom att klicka på den i listan så att jag snabbt kommer till editorn."

Acceptance Criteria:

- Klick på en fil i listan navigerar till editorn med filen laddad
- Fungerar för både ägda och delade filer
- Felmeddelande visas om filen inte kan laddas
- Användaren kan ta sig tillbaka till fillistan från editorn

Hur skriver man bra AC?

- **Var specifik** — undvik "fungerar korrekt" och liknande
- **En sak i taget** per kriterium
- **Tänk på edge cases** — tom lista, fel lösenord etc
- **Använd "givet/när/då"** när det underlättar:
  - **givet** att användaren inte är inloggad
  - **när** de försöker öppna fillistan
  - **så** omdirigeras de till inloggningssidan.

Tasks

Vad är en task?

Det finns inget facit nu heller :(

> *en task är tillräckligt bra när den som ska göra den förstår vad som förväntas och den som estimerar den kan sätta ett tal på det utan att behöva gissa.*

- En task är **det ni faktiskt ska koda**;
  - helst tillräckligt liten för att en person ska kunna slutföra den på under en dag.
  - tillräckligt avgränsad för att kunna granskas i ett enskilt PR.
- Ett bra sätt att identifiera tasks är att fråga sig :
  - vad är det första jag behöver göra för att storyn ska fungera?
  - T ex att skapa en komponent
  - hämta data från API
  - hantera ett felfall

forts. Vad är en task?

Tips:

- Separera vad som visas/renderas i visuella vyer, från var data hämtas eller sparas.
  Gör separata tasks för det istället.
- Man kan använda hårdkodad data först för ett slags 'proof-of-concept'. Koppla sedan på API:t i en separat task. Det kan göra stegen lättare att bygga ut och felsöka.
- En task som känns för stor — mer än åtta timmar — är ett tecken på att den kan delas upp.
- En task som känns osäker att estimera är ofta ett tecken på att en spike behövs först.

Namngivning av tasks

Så att de kan härledas till en US:

`T-[US-nummer]-[löpnummer]`

Exempel tasks från US-101

> **US-101 — "Som inloggad användare vill jag se en fillista med metadata..."**

T-101-1 · Skapa FileList-komponent · 3h · Emil
Bygger grundstrukturen. Ingen data ännu, bara komponenten med rätt props-interface.

T-101-2 · Hämta filer från API · Emil
Kopplar mot `GET /api/files`. Hanterar laddning, tomt svar och felfall.

T-101-3 · Rendera metadata per fil · Emil
Visar titel, språk, senast ändrad och skapare. Formaterar datum läsbart.

T-101-4 · Visuell markering av delade filer · Masse
Lägger till ikon eller badge för delade filer.

T-101-5 · Skydda routen — omdirigera ej inloggad · Masse
Säkerställer att `/files` kräver autentisering.

Exempel tasks från US-102

> **US-102 — "Som inloggad användare vill jag kunna öppna en fil genom att klicka..."**

T-102-1 · Navigera till editor vid klick · 2h · Emil
Klick på fil navigerar till `/editor/:fileId` med rätt fil laddad i Monaco Editor.

T-102-2 · Felhantering vid filöppning · 2h · Emil
Visar felmeddelande vid 404 eller 403. Användaren kan ta sig tillbaka till listan.

Estimering

Poängen med estimering är inte att träffa perfekt — utan att **tänka igenom arbetet i förväg**.

En sprint kan aldrig innehålla mer arbete än vad gruppen faktiskt har timmar till.

Ange estimat per task, summera per sprint — och håll er inom budgeten.

Ett sätt att estimera är att använda planning poker;

scrumpoker online

planning poker online

poki poki

Definition of Done (DoD)

DoD är teamets **gemensamma överenskommelse** om vad som krävs för att en uppgift ska anses färdigställd.

Det är en **objektiv checklista** som gäller för *alla* arbetsuppgifter i projektet — inte bara en specifik story.

> *DoD minskar tvetydighet och gör code reviews konfliktfria.*
> *Om något saknas i en PR — returnera den.*

Exempel DoD

Ett exempel på DoD skulle kunna vara:

- Koden fungerar lokalt utan fel
- Funktionen uppfyller sin stories Acceptance Criteria
- Felhantering finns för rimliga felfall
- Koden är granskad och godkänd av en annan gruppmedlem (PR)
- PR:en är mergad till `main`
- README är uppdaterad om det behövs

AC kontra DoD

Acceptance Criteria

- Specifik för **EN** user story
- Fokus: *funktionella krav*
- Fråga: *Gör funktionen rätt sak enligt användaren?*

Definition of Done

- Global för **ALLA** uppgifter
- Fokus: *kvalitet och process*
- Fråga: *Är koden granskad, testad och mergad enligt våra regler?*

> *AC bekräftar att ni har **byggt rätt funktion**.*
> *DoD bekräftar att ni har **byggt funktionen på rätt sätt**.*

US, Tasks, AC och DoD i arbetsflödet

1. **Sprint Planning** — Analysera utifrån kravspec eller välj stories från eventuell backlog, bryt ned till tasks, estimera, håll er inom kapacitet.
2. **Varje dag** — Koda tasks, öppna PR när task är klar
3. **Code review** — Granska mot AC + DoD-checklistan
4. **Sprint Review** — Visa vad som är klart (= uppfyller AC + DoD)
5. **Sprint Retrospective** — Förbättra processen, justera DoD vid behov

Sammanfattning

Begrepp	Tillhör	Fråga det svarar
User Story	Product Backlog	Vad ska byggas och för vem?
Acceptance Criteria	User Story	Hur vet vi att storyn är klar?
Task	Sprint Backlog	Vad ska jag koda härnäst?
Estimat	Task	Ryms det i sprinten?
Definition of Done	Hela projektet	Håller koden tillräcklig kvalitet?

Kravspecen är startpunkten.

Tack & förlåt